Robot CI: A Continuous Regression Testing Framework for Robotic Manipulation Policies

Abstract-Robotic manipulation policies increasingly evolve through retraining, fine-tuning, and architectural revision, making version-to-version behavioral regression an important testing problem. A candidate policy can retain task success while requiring more execution steps, incurring additional collisions, or accumulating lower reward - degradation that binary success/failure evaluation cannot detect. Existing robotic testing and policy-evaluation research separately establishes simulation-based testing, continuous-integration practice for robot software, and richer multi-metric manipulation benchmarks, but these capabilities are generally presented as independent workflows. This paper presents Robot CI, a software-oriented continuous regression-testing framework that treats manipulation policies as versioned artifacts. The current implementation uses PyBullet as the physics backend and combines policy version management, a reusable scenario library, an automated closed-loop rollout engine, multi-metric telemetry, configurable regression detection, persistent experiment storage, and a Streamlit dashboard. A controlled comparison of a baseline and a candidate policy over five manipulation scenarios shows that the candidate required more completion steps in all five scenarios, incurred 22 total collisions concentrated in the two hardest pick-and-place tasks (versus none for the baseline), and had lower cumulative reward in four of five scenarios. These findings demonstrate that success-only evaluation can conceal meaningful behavioral degradation. The present study is limited to single-run PyBullet experiments; multi-seed statistical validation and physical-robot evaluation remain future work.

Index Terms-robot policy regression testing, robotic manipulation, simulation-based evaluation, PyBullet, continuous integration, behavioral testing, telemetry, policy evaluation, sim-to-real transfer, benchmark evaluation.

I. INTRODUCTION

Learned robotic manipulation policies are increasingly developed and maintained as versioned software artifacts rather than fixed programs: a single project can produce many policy versions as training data is expanded, hyperparameters are retuned, architectures are revised, or demonstrations are added. Testing this software is difficult because physical execution is expensive, hard to reproduce, and constrained by hardware access, sensor noise, and safety considerations [1], [2]. Simulation offers a lower-cost, repeatable substitute, and prior work has already applied continuous-integration (CI) practice to robot models and controllers [25], [26].

Even when a policy is evaluated carefully, a binary task-success metric is an impoverished signal. A candidate policy may still reach the goal state - and therefore be recorded as "successful" - while requiring more time steps, colliding with the environment more often, or accumulating lower reward than a trusted baseline. Recent policy-evaluation work argues explicitly for richer experimental reporting, including documented conditions, complementary metrics, statistical analysis, and qualitative failure description [42], and RoboEval demonstrates empirically that policies with comparable success rates can differ substantially in efficiency, coordination, and failure behavior [41]. None of these behavioral differences is visible in a success rate alone, yet each is the kind of change a mature software-engineering process would flag automatically before a release.

This is the research gap addressed in this paper: the literature has separately studied simulation-based policy evaluation, robotics-specific CI practice, and software/ML regression testing (Section II), but comparatively little existing work - to the best of our knowledge, based on the literature search conducted for this paper - integrates these threads into a single, versioned, continuously repeatable regression-testing workflow purpose-built for learned manipulation policies, in which a baseline and a candidate are rolled out under matched simulated conditions, multi-dimensional telemetry is collected, behavioral regressions are flagged, and results are persisted and visualized for engineering decision-making.

We present Robot CI, a framework that treats a robotic manipulation policy as a versioned software artifact and evaluates a baseline policy against a candidate policy under matched, reproducible conditions. The current implementation uses PyBullet [3] as the physics backend. Robot CI standardizes scenarios, executes closed-loop rollouts of both policies, records multi-dimensional telemetry, computes behavioral metrics, flags regressions using configurable thresholds, persists results, and exposes all of the above through a Streamlit dashboard. Migration to higher-fidelity simulators such as Isaac Sim is discussed only as future work (Section IX) and is not part of the present experimental setup.

The contributions are, stated conservatively: (1) a software-oriented CI architecture for versioned robotic manipulation policies; (2) a reproducible baseline-versus-candidate simulation workflow under matched scenario conditions; (3) a multi-metric, telemetry-driven approach to behavioral regression detection beyond binary task success; (4) persistent, scenario-level experiment reporting; and (5) an interactive dashboard for interpreting regression outcomes. Physical-robot and sim-to-real validation are not claimed as completed contributions of the present study.

II. RELATED WORK

A. Robotic Testing, Verification and Validation

Araujo et al. [1] performed a systematic literature review of testing, validation, and verification techniques for robotic and autonomous systems, screening more than ten thousand candidate studies down to 195 included papers and organizing the field around physical testing, model-based testing, runtime monitoring, and formal verification. Their review observes that most existing testing techniques are evaluated in isolation on individual case studies rather than integrated into a continuous, versioned evaluation workflow - the gap Robot CI targets from a software-engineering rather than a formal-methods perspective. Complementing this, Luckcuck et al. [2] surveyed formal specification and verification for autonomous robotics, arguing that testing and simulation alone cannot certify correctness for safety-critical autonomy. Robot CI does not attempt formal verification; it instead adopts the pragmatic, empirical stance common to software regression testing, in which repeated, reproducible behavioral comparison - rather than proof - is used to catch unintended changes between policy versions.

B. Simulation-Based Robot Policy Evaluation

Physics simulators are the substrate on which reproducible, parallelizable robot-policy evaluation is built. PyBullet [3], the backend used by the current Robot CI implementation, is a widely used open-source physics engine for articulated-body simulation, robotics, and machine-learning research. Simulation frameworks such as robosuite [4], built on a modular task-and-controller abstraction, and RLBench [5], a large-scale manipulation benchmark and learning environment, standardize task definitions so that policies can be compared under identical simulated conditions. Li et al. [6] proposed SIMPLER, showing that carefully constructed simulation environments can serve as a scalable and reliable proxy for real-world manipulation-policy evaluation, provided that control and visual disparities between simulation and reality are explicitly modeled. Robot CI adopts the same underlying premise - that matched, reproducible simulated rollouts are a legitimate evaluation substrate - but applies it to baseline-versus-candidate comparison rather than to absolute real-world-performance prediction.

C. Automated and Regression Testing for Robotics

A substantial and largely independent body of work has applied continuous-integration and regression-testing practice to robotic software. Biggs [23] presents an early case study on regression testing of individual robot hardware-interaction components in a repeatable fashion, and Bihlmaier and Wörn [24] introduce Robot Unit Testing, using simulation to bridge automated software checks and time-consuming manual hardware tests. Lier et al. [25] propose a CI approach that iteratively validates simulated robot models against a real robot, and Mossige et al. [26] describe an industrial constraint-programming CI pipeline, deployed at ABB Robotics, that automatically generates and executes test sequences for industrial robot controllers. Ernits et al. [27] and Araiza-Illan et al. [28] apply model-based test generation - from topological maps and from automata/BDI-agent models, respectively - to automate scenario coverage for ROS-based navigation and manipulation software. Honfi et al. [29] address model-based regression test selection for autonomous robots directly, and Santos et al. [30] and Abdelgawad et al. [31] apply property-based and model-based testing to ROS communication contracts and real-time motion planning, respectively.

Closer to the CI/CD framing of Robot CI, Jiang et al. [32] report a Jenkins/Docker-based continuous-integration and testing

framework for autonomous-racing controllers evaluated in the CARLA simulator, and Vieira et al. [39] describe a GitHub Actions/AWS RoboMaker cloud-simulation CI/CD pipeline for calibrating a ROS-based robotic system. Afzal et al. [33] survey robotics developers on how simulation is used for test automation in practice, and Yayan [34] and Qu et al. [35] apply mutation-based fault injection and CI/CD-integrated reinforcement-learning deployment, respectively, to ROS-compatible and robotic-IoT software. Sohail et al. [36] use property-based testing in simulation to verify correct robot action execution during tabletop manipulation, and Nguyen et al. [38] extend behavior-driven development to specify and automatically verify acceptance criteria for pick-and-place applications in Isaac Sim. Jiang et al. [40] provide the most comprehensive synthesis of this thread, systematically reviewing 122 studies on regression-test prioritization, minimization, and selection for ROS-based autonomous systems and identifying regression testing for such systems as a comparatively underexplored area. Closest in spirit to Robot CI's baseline-versus-candidate philosophy is Andersson and Carlstedt [37], who compare successive industrial robot-controller versions in simulation using a Hausdorff-distance trajectory-similarity test to detect motion anomalies seeded as artificial bugs.

Table I positions Robot CI against this body of work along the dimensions that most directly bear on its design: whether the system under test is a classical/rule-based controller or a learned policy, whether comparison is single-metric or multi-metric, and whether the workflow is a one-off study or a reusable, versioned CI-style pipeline. Almost all prior robotics-CI work targets classical or single-controller software and, with the partial exception of [37], compares versions using a single behavioral signal rather than a multi-dimensional telemetry suite.

TABLE I. COMPARISON OF ROBOT CI WITH CLOSELY RELATED ROBOTICS CI/REGRESSION-TESTING WORK

Prior Work	Target System	Comparison Basis	Multi-Metric?	Versioned CI Workflow?
Biggs [23]; Mossige et al. [26]	Classical robot control/hardware software	Pass/fail, fault detection	No	Component-level, not policy-oriented
Honfi et al. [29]	Autonomous robot software models	Model-based test selection	No	Selection strategy, not full pipeline
Andersson & Carlstedt [37]	Industrial robot controller versions	Trajectory similarity (Hausdorff distance)	Single metric	Version-to-version, not multi-metric
Jiang et al. [32]; Vieira et al. [39]	Autonomous/ROS controller software	Simulated CI/CD test execution	No	Yes (classical software)
RoboEval [41]	Learned manipulation policies	Structured benchmark metrics	Yes	Benchmark, not versioned CI
SureSim [45]	Learned manipulation policies	Sim + real statistical inference	Yes (statistical)	Evaluation study, not CI pipeline
Robot CI (this paper)	Learned manipulation policies	Baseline-vs-candidate telemetry	Yes	Yes

D. Machine-Learning Regression Testing

Robot CI's core methodology - comparing a baseline artifact against a candidate artifact under matched conditions to detect unintended behavioral change - is a direct application of software regression-testing concepts. Rothermel and Harrold [11] established foundational techniques for regression test selection, and Yoo and Harman [12] survey minimization, selection, and prioritization strategies for regression test suites in traditional software. Zhang et al. [13] extend this line of work to machine-learned software, surveying testing properties, workflows, and open problems for ML systems, and Riccio et al. [14] provide a systematic mapping of testing techniques for ML-based systems more broadly. Zhu et al. [15] address regression testing specifically for ML-based systems, proposing automated authoring of regression tests that detect prediction inconsistencies between model versions - a problem structurally analogous to the behavioral-metric regressions Robot CI targets for robot policies. At the operational level, Kreuzberger et al. [16] describe MLOps architectures for continuously integrating, testing, and deploying ML artifacts, providing the software-lifecycle context into which a framework such as Robot CI is intended to fit.

E. Manipulation Policy Benchmarking and Multi-Metric Evaluation

A separate strand of recent work argues directly that manipulation-policy evaluation should move beyond a single success rate. Kress-Gazit et al. [42] argue that robot-learning evaluation should be treated as an empirical science and should explicitly document experimental conditions, complementary metrics, statistical analysis, and failure modes rather than reporting success rate alone. RoboEval [41] operationalizes this broader perspective with structured, bimanual manipulation tasks and diagnostic metrics for efficiency, safety/stability, and progress, showing that policies with comparable success rates can differ substantially in execution quality. Statistical rigor is a further consideration: Vincent et al. [43] develop confidence-based evaluation for behavior-cloning policies under limited rollout budgets and demonstrate policy comparison in both simulation and hardware, and Snyder et al. [44] extend this direction with sample-efficient, statistically rigorous policy comparison across multiple performance measures, including reward and trajectory-related quantities. Badithela et al. [45] similarly address the problem of combining large-scale simulation with limited physical evaluation - the SureSim approach - to obtain statistically reliable real-world inferences from imperfect simulators. Robot CI adopts the same multi-metric philosophy as this literature but frames it as a versioned, dashboard-backed regression-testing workflow rather than a benchmark suite or a statistical-inference method in isolation; it does not yet claim the confidence-interval or effect-size guarantees reported in [43]–[45] for its current five-scenario experiment.

F. Statistical Reliability and Reproducibility

The reliability of policy comparisons is itself an active research concern. Henderson et al. [17] show that many reported improvements in deep reinforcement learning are not distinguishable from noise once implementation details, random seeds, and evaluation protocol are accounted for, and Agarwal et al. [18] argue for reporting stratified bootstrap confidence intervals rather than point estimates when comparing reinforcement-learning agents. Lynnerup et al. [19] study reproducibility specifically for real-world robot reinforcement learning, echoing the same concerns in a physical setting, and Voloshin et al. [20] empirically study off-policy evaluation methods, which are relevant to policy selection when environment interaction is costly. At the level of detecting

behavioral inconsistency in learned systems more generally, Tian et al. [21] (DeepTest) and Zhang et al. [22] (DeepRoad) apply metamorphic and transformation-based testing to autonomous-driving neural networks, showing that systematic, automated perturbation of test conditions can reveal inconsistent behavior that accuracy metrics alone do not expose. Robot CI's regression-detection module is conceptually aligned with this literature - multiple behavioral metrics, evaluated across matched scenarios, are used in place of a single aggregate score - but effect-size and confidence-interval-based statistical evidence, in the sense advocated by [17], [18], has not yet been applied to the five-scenario experiment reported in Section VI, which should be read as a controlled illustrative comparison rather than a statistically powered study.

G. Sim-to-Real Transfer

Because Robot CI's current results are obtained entirely in simulation, the eventual transfer of its findings to physical hardware is informed by the sim-to-real transfer literature. Zhao et al. [7] survey approaches to closing the sim-to-real gap in deep reinforcement learning for robotics and identify domain randomization as the most widely adopted technique. Tobin et al. [8] and Peng et al. [9] demonstrate that randomizing visual appearance and physical dynamics parameters, respectively, during simulated training improves transfer to real hardware without requiring a high-fidelity digital twin, and Muratore et al. [10] review the broader family of randomized-simulation techniques for sim-to-real transfer. These works motivate Robot CI's planned sim-to-real gap-analysis stage (Section IX); no domain randomization or sim-to-real discrepancy measurement has yet been performed, and no such results are reported in this paper.

H. Research Gap

The reviewed literature establishes: (i) that robotic systems require dedicated testing and verification methodologies beyond those used for conventional software [1], [2]; (ii) that simulation can serve as a scalable and, under the right conditions, reliable proxy for policy evaluation [3]–[6]; (iii) that a mature, largely independent robotics-CI literature already applies continuous-integration and regression-testing practice to classical robot control software, with only limited and recent extension to learned, multi-metric manipulation-policy evaluation [23]–[40]; (iv) that machine-learned software benefits from regression-testing concepts adapted from traditional software engineering, together with statistically disciplined evaluation protocols [11]–[22]; (v) that recent manipulation-evaluation work argues explicitly for multi-metric, statistically grounded policy comparison [41]–[45]; and (vi) that sim-to-real transfer requires explicit treatment of the reality gap once physical validation is attempted [7]–[10]. The identified studies, however, generally address one part of this pipeline in isolation. To the best of our knowledge, based on the literature search conducted for this paper, comparatively little existing work integrates versioned policy artifacts, matched baseline/candidate simulation rollouts, multi-dimensional telemetry, multi-metric behavioral regression detection, persistent experiment reporting, and an interactive dashboard into a single, reusable, CI-style workflow purpose-built for learned manipulation policies.

This integration - rather than any single technique within it - is the contribution Robot CI proposes. The contribution is consequently not a new physics engine or a new learning algorithm; it is the combination of policy versioning, matched scenarios, automated rollout telemetry, behavioral regression detection, result persistence, and dashboard-based inspection into a repeatable engineering workflow. This claim is made cautiously: it reflects the scope of the literature search described above rather than an exhaustive systematic review, and a broader review is recommended before the claim is repeated in a venue with a formal novelty requirement.

III. PROBLEM FORMULATION

Baseline policy. A baseline policy π_b is a previously accepted, versioned control policy that maps an observation o_t to an action $a_t = \pi_b(o_t)$, against which subsequent policy versions are compared.

Candidate policy. A candidate policy π_c is a newer, unreleased version of the same policy family - for example, retrained with additional data, fine-tuned hyperparameters, or a revised architecture - whose behavioral quality relative to π_b is the subject of evaluation.

Scenario. A scenario $s \in S$ is a reproducible, parameterized simulation configuration - initial robot and object configuration, target configuration, simulation conditions, step limit, and random seed - that fully determines the initial conditions of an episode so that π_b and π_c can be rolled out under matched conditions.

Rollout. A rollout $\tau = (o_0, a_0, o_1, a_1, \ldots, o_T)$ is the closed-loop trajectory produced by executing a policy π in scenario s inside the PyBullet simulation backend until task completion, task failure, or the step limit T is reached.

Performance metrics. For a rollout τ, a metric vector $M(\tau)$ is computed by the evaluation engine (Section IV-F), comprising at minimum task success, completion steps, collision count, and cumulative reward, together with any trajectory- and action-based metrics the telemetry layer supports for that scenario.

Regression. Given matched rollouts τ_b and τ_c of π_b and π_c on the same scenario s, a regression is a threshold-significant degradation in one or more components of $M(\tau_c)$ relative to $M(\tau_b)$, irrespective of whether π_c still achieves task success on s. For metrics where lower values are better (completion steps, collisions), an increase is a degradation; for reward, a decrease is a degradation. This definition deliberately decouples "regression" from "failure": a candidate that succeeds but degrades is still classified as regressed on the affected metric. The current experiment reports raw metric differences rather than statistically or threshold-classified verdicts, because specific threshold values and repeated-trial statistics were not part of the supplied five-scenario result set.

IV. ROBOT CI FRAMEWORK

Fig. 1 summarizes the implemented pipeline: policy version → scenario selection → PyBullet simulation → policy rollout → telemetry collection → behavioral evaluation → regression detection → result storage → Streamlit dashboard. Each stage is described below together with its current implementation status; all nine components listed are implemented in the current system.

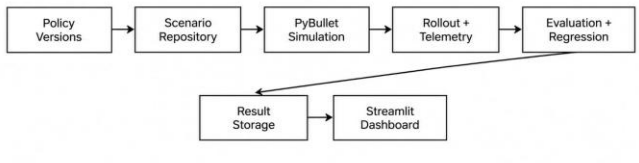


Fig. 1. Robot CI architecture for versioned manipulation-policy regression testing.

A. Policy Version Management

The Policy Manager maintains a registry of policy versions with configuration metadata sufficient to reproduce a given version's behavior. It designates one version as the baseline and one as the candidate for a given comparison run and enforces that both are evaluated against identical scenario configurations.

B. Scenario Management

The Scenario Framework provides a library of reusable, parameterized test scenarios - pick_and_place_easy, pick_and_place_medium, pick_and_place_hard, reach_diagonal, and reach_simple - with deterministic, reproducible generation so that the same scenario instance can be replayed for both the baseline and candidate policy.

C. PyBullet Simulation

PyBullet [3] is the current physics simulation backend. It executes rigid-body dynamics, collision detection, and contact response for the robot and manipulated objects, and exposes the state information (joint states, end-effector pose, contact points) required by the telemetry layer. The current implementation and the results reported in Section VI use PyBullet exclusively; no Isaac Sim or Isaac Lab experiment has been run at the time of writing.

D. Automated Rollout Engine

The rollout engine executes a policy in closed loop: an observation is read from the simulator, passed to the policy, the resulting action is applied to PyBullet, the simulation is advanced by one step, the new state is obtained, and telemetry is recorded. This loop repeats until task completion, task failure, or the configured step limit is reached.

E. Telemetry Collection

The telemetry layer records, per simulation step, the observation, the action, the instantaneous reward, end-effector X/Y/Z position, target distance, action magnitude, collision information, and termination status. This per-step record is the raw material from which all downstream behavioral metrics are computed.

F. Evaluation Engine

The evaluation engine aggregates telemetry into scenario-level metrics. Equations (1)–(5) formalize the metrics computed for the reported experiment; equations (6)–(7) define trajectory- and action-based metrics that the telemetry layer supports but that were not computed for the five-scenario result set discussed in Section VI, since only aggregate step, collision, and reward values were available for that experiment. Symbols follow Section III.

$$S = N_success \,/\, N_trials \tag{1}$$

$$\bar{T} = (1/N)\, \Sigma_i\, T_i \tag{2}$$

$$\bar{C} = N_collisions \,/\, N_trials \tag{3}$$

$$\Delta R = R_candidate - R_baseline \tag{4}$$

$$\Delta T = T_candidate - T_baseline \tag{5}$$

$$MSE = (1/T)\, \Sigma_t\, \|x_c(t) - x_b(t)\|^2 \tag{6}$$

$$RMSE_action = \sqrt{[(1/T)\, \Sigma_t\, \|a_c(t) - a_b(t)\|^2]} \tag{7}$$

S is the success rate over N_trials matched rollouts of a given policy on a given scenario or scenario set; $\bar{T}$ is the mean completion-step count; $\bar{C}$ is the mean collision count (or collision-occurrence rate) per trial; ΔR and ΔT are the per-scenario reward and completion-step differences reported in Section VI. Equations (6) and (7) define time-aligned trajectory error and action-magnitude difference between candidate and baseline rollouts; the telemetry layer logs the end-effector position and action vectors needed to compute them, but they are reported here as implemented, telemetry-supported metrics proposed for future experimental runs rather than as results already obtained.

G. Regression Detection

The candidate policy is compared against the baseline policy using the metrics of Section IV-F. The system flags a regression when a metric degrades beyond a configurable, per-metric and per-scenario threshold, so that detection does not rely solely on a binary success/failure signal. Beyond raw threshold comparison, the framework is designed to support evidence-based regression classification using scenario consistency across repeated trials, statistical evidence (e.g., confidence intervals in the style of [18]) where multiple-seed data is available, and effect-size measures; in the current five-scenario experiment, only single-trial metric degradation is available, and statistical- and effect-size-based classification are marked as future work (Section IX).

H. Result Storage

Experiment configurations, telemetry, computed metrics, and regression-detection outcomes are persisted for reproducibility and later analysis.

I. Streamlit Dashboard

A Streamlit dashboard provides an experiment overview, baseline/candidate comparison views, scenario-level metric visualizations, regression-outcome displays, and experiment history, giving engineers an interactive interface for interpreting comparison outcomes without inspecting raw telemetry directly.

V. EXPERIMENTAL METHODOLOGY

A. Experimental Setup

The reported experiment compares one baseline policy π_b with one candidate policy π_c over five scenarios, using the components of Section IV. Exact hardware specification, PyBullet and library versions, and the random seed used for the reported run are recorded in the project's experiment log rather than restated here; no numeric values for these fields are invented in this paper. PyBullet is configured with rigid-body simulation of the robot arm and manipulated objects, collision detection enabled, and a fixed physics time step; solver iterations and remaining physics parameters follow the project's default configuration.

B. Policy Versions

π_b and π_c are drawn from the same policy family and are identified by version metadata in the Policy Manager (Section IV-A). The specific training-data and checkpoint differences between the two versions are recorded alongside the experiment log.

C. Scenario Set

Five scenarios of increasing difficulty and task type were used: pick_and_place_easy, pick_and_place_medium, pick_and_place_hard, reach_diagonal, and reach_simple, as defined by the Scenario Framework (Section IV-B).

D. Rollout Procedure

Each policy was rolled out in closed loop on each scenario as described in Section IV-D, with telemetry recorded at every simulation step until task completion, failure, or the step limit was reached. The supplied result set contains one rollout per policy per scenario.

E. Evaluation Metrics

Completion steps, collision count, and cumulative reward were recorded for each policy on each scenario (Table II). Success/failure was recorded but is not itemized numerically in the source data; all five scenarios are treated as completed by both policies for the purpose of this analysis, consistent with the framing that a regression can occur even when a task is nominally completed.

F. Regression Criteria

Regression thresholds are configurable per metric and per scenario (Section IV-G); no specific threshold values were supplied for this experiment, so Section VI reports raw metric differences (Δsteps, Δcollisions, Δreward) rather than a threshold-classified pass/fail verdict.

VI. RESULTS

Table II reproduces, without alteration, the controlled comparison of the baseline and candidate policies across five scenarios, as provided by the project's experiment output. All values are single-run, scenario-level aggregates; no repeated trials, seeds, or confidence intervals are available, and no additional scenarios or policies beyond those shown were evaluated. Table III presents the derived per-scenario differences, computed directly from Table II using equations (4)–(5).

TABLE II. BASELINE (B.) VS. CANDIDATE (C.) POLICY - FIVE-SCENARIO COMPARISON

Scenario	B. Stp	B. Col	B. Rew	C. Stp	C. Col	C. Rew
P&P Easy	21	0	6.59	28	0	6.89
P&P Medium	32	0	3.77	37	9	-4.84
P&P Hard	48	0	-1.40	55	13	-13.82
Reach Diagonal	13	0	6.67	36	0	5.02
Reach Simple	6	0	9.23	12	0	8.47

TABLE III. DERIVED SCENARIO-LEVEL REGRESSION INDICATORS

Scenario	Δ Stp	Δ Stp %	Δ Col	Δ Rew	Degraded?
P&P Easy	+7	+33.3%	+0	+0.30	No (reward ↑)
P&P Medium	+5	+15.6%	+9	-8.61	Yes
P&P Hard	+7	+14.6%	+13	-12.42	Yes
Reach Diagonal	+23	+176.9%	+0	-1.65	Yes
Reach Simple	+6	+100.0%	+0	-0.76	Yes

A. Completion-Step Analysis

The candidate required more completion steps than the baseline in all five scenarios, with increases ranging from +14.6% (pick_and_place_hard) to +176.9% (reach_diagonal). The reach_diagonal result is particularly illustrative: both policies completed the task in the supplied result framing, but the candidate required 36 steps instead of 13, nearly tripling the step count while incurring zero collisions in either policy. A success-only gate would therefore fail to expose this trajectory-efficiency regression.

B. Collision Analysis

The baseline recorded zero collisions in all five scenarios. The candidate also recorded zero collisions in pick_and_place_easy, reach_diagonal, and reach_simple, but incurred nine collisions in pick_and_place_medium and thirteen in pick_and_place_hard - 22 collisions in total. This is the most severe behavioral regression observed in the experiment, and it is concentrated in the two hardest pick-and-place scenarios, suggesting that the candidate's degradation is tied to tighter manipulation constraints rather than being uniform across task types.

C. Reward Analysis

Candidate reward decreased in four scenarios: −8.61 for pick_and_place_medium, −12.42 for pick_and_place_hard, −1.65 for reach_diagonal, and −0.76 for reach_simple. Only pick_and_place_easy showed a reward increase (+0.30), despite requiring seven additional steps. This illustrates why individual metrics can disagree in direction, and why a multi-metric regression report is preferable to a single acceptance score.

D. Scenario-Level Regression Findings

Across the five scenarios, the candidate is best characterized as completing every scenario while exhibiting a consistent completion-step regression and scenario-dependent collision and reward regressions concentrated in the medium- and hard-difficulty pick-and-place tasks. Under the regression definition of Section III, the candidate is regressed on at least one metric in four of the five scenarios - all except pick_and_place_easy, where only the completion-step metric regresses while reward improves. The result should not be interpreted as a statistically general conclusion about the policy family, since the experiment contains one baseline/candidate pair and one rollout per scenario; it does, however, demonstrate the type of behavioral change the Robot CI pipeline is designed to surface.

VII. DISCUSSION

A. Success-Only Evaluation Is Insufficient

The reach_diagonal result is the clearest illustration of this paper's central motivation: the candidate completed the task in every recorded rollout, so a success-only acceptance test would have shown no difference between baseline and candidate on this scenario, yet the candidate required 176.9% more completion steps to do so. This is consistent with the motivation shared by recent multi-metric policy-evaluation studies [41], [42], and directly supports the case for behavioral, rather than binary, regression detection.

B. Multi-Metric Regression Detection and Relationship to Existing CI Testing

Robot CI inherits the baseline-versus-candidate mindset of software regression testing [11], [23], [26], [29], but applies it to a learned, closed-loop manipulation policy rather than to a component-level or requirement-level test target. Property-based manipulation testing [36] provides a complementary precedent for automatically checking robot behavior across generated scenarios, and DeepRIoT [35] demonstrates the feasibility of integrating learned robotic controllers into CI/CD workflows. Treating policy comparison as a regression-testing problem - with versioned artifacts, matched scenarios, and multi-metric evaluation - shifts the question asked of a new policy version from "does it still work?" to "does it still work at least as well as before, on every metric that matters?", without requiring additional physical-robot time for every candidate.

C. Reproducibility and Scalability

Reproducibility in Robot CI rests on three pillars: deterministic, versioned scenarios (Section IV-B); explicit policy versioning (Section IV-A); and persistent storage of telemetry, metrics, and configuration (Section IV-H). Full numerical reproducibility of the specific results in Section VI additionally requires the recorded random seed and environment details noted in Section V-A. Because rollouts in PyBullet are comparatively inexpensive relative to

physical-robot trials, the same workflow can, in principle, be extended to a substantially larger scenario corpus and to repeated trials per scenario without additional hardware time - a prerequisite for the statistically powered evaluation identified as future work in Section IX. Afzal et al. [33] identify simulation as a promising basis for test automation while also documenting practical challenges that must be addressed in real development workflows.

D. Practical Engineering Value

The concentration of collision and reward regressions in the two hardest pick-and-place scenarios, contrasted with the pure trajectory-efficiency regression in reach_diagonal, suggests that the candidate policy's behavioral changes are not uniform across task types. This kind of scenario-conditioned pattern is precisely what a single aggregate success rate would obscure, and it is the type of finding a scenario-level regression report is intended to surface for an engineer deciding whether to accept a candidate checkpoint.

VIII. LIMITATIONS

- The reported experiment contains five scenarios and one rollout per policy per scenario; metric variance is therefore unknown, and observed differences cannot be attributed to the policy version alone with statistical confidence.
- No confidence intervals, hypothesis tests, or effect-size estimates are reported for the five-scenario experiment.
- Trajectory-error (MSE) and action-RMSE metrics (equations (6)–(7)) are supported by the telemetry design but were not computed for the five-scenario result set.
- The regression-analysis module's statistical-evidence and effect-size classification capability (Section IV-G) has not yet been exercised.
- The present results use PyBullet exclusively and do not establish physical-robot or sim-to-real validity; no domain randomization or sim-to-real discrepancy measurement has been performed.
- The study evaluates one baseline/candidate pair and should not be generalized to other policy versions without additional experiments.
- Exact environment details (PyBullet version, physics parameters, random seed) should be finalized from the project's experiment log before submission to a venue with strict reproducibility requirements.

IX. FUTURE WORK

- Physical SO-101 validation: executing the same baseline/candidate comparison protocol on physical hardware to obtain real success, collision, and completion-step measurements.
- Real telemetry: extending the telemetry layer to log the same fields (end-effector pose, action magnitude, collisions, termination) from physical-robot sensors and controllers.
- Sim-to-real discrepancy analysis: quantifying the gap between PyBullet-simulated and physical-robot metrics using the methodology surveyed in Section II-G [7]–[10], once real measurements are available.
- Multi-seed evaluation and confidence intervals for scenario-level regression decisions, and expansion to a larger, systematically varied scenario corpus to reduce sampling variance.
- Integration of statistical policy-comparison methods for early stopping and uncertainty-aware regression decisions [43], [44].
- Evaluation of additional candidate-policy versions per baseline, to study regression trends across a version history rather than a single baseline/candidate pair.
- CI/CD integration, consistent with MLOps practice [16] and robotics-CI precedent [26], [32], [39], so that a newly registered policy checkpoint automatically triggers the regression suite.
- Comparison with additional simulation backends, such as NVIDIA Isaac Sim/Isaac Lab, after the PyBullet pipeline is experimentally established; no Isaac Sim experiments have been conducted at the time of writing.

X. CONCLUSION

This paper presented Robot CI, a software-oriented continuous regression-testing framework for robotic manipulation policies. The current implementation treats policy versions as testable artifacts and combines policy version management, a reusable scenario library, PyBullet-based closed-loop rollouts, telemetry collection, multi-metric evaluation, regression detection, persistent result storage, and dashboard-based reporting; all nine components are implemented and were exercised in a controlled, five-scenario comparison. In the reported experiment, the candidate policy required more completion steps in every scenario, incurred 22 total collisions concentrated in the two hardest pick-and-place scenarios where the baseline had none, and reduced cumulative reward in four of five scenarios. These findings demonstrate that a policy can exhibit meaningful behavioral degradation without being identified by a success-only evaluation protocol, and they support the case for versioned, multi-metric regression testing as a complement to conventional acceptance testing in robot-learning workflows. The present results are limited to single-run simulation evidence over one baseline/candidate pair; statistical multi-seed analysis and physical validation are the necessary next steps toward a fully validated Robot CI pipeline.

REFERENCES

[1] H. Araujo, M. R. Mousavi, and M. Varshosaz, "Testing, Validation, and Verification of Robotic and Autonomous Systems: A Systematic Review," ACM Trans. Softw. Eng. Methodol., vol. 32, no. 2, Art. 51, 2023.

[2] M. Luckcuck, M. Farrell, L. Dennis, C. Dixon, and M. Fisher, "Formal Specification and Verification of Autonomous Robotic Systems: A Survey," ACM Comput. Surv., vol. 52, no. 5, 2019.

[3] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016–2021. [Online]. Available: http://pybullet.org

[4] Y. Zhu, J. Wong, A. Mandlekar, and R. Martín-Martín, "robosuite: A Modular Simulation Framework and Benchmark for Robot Learning," arXiv:2009.12293, 2020.

[5] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "RLBench: The Robot Learning Benchmark & Learning Environment," IEEE Robot. Autom. Lett., vol. 5, no. 2, pp. 3019–3026, 2020.

[6] X. Li, K. Hsu, J. Gu, O. Mees, K. Pertsch, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao, "Evaluating Real-World Robot Manipulation Policies in Simulation," in Proc. 8th Conf. Robot Learning (CoRL), PMLR, vol. 270, 2025, pp. 3705–3728.

[7] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey," in Proc. IEEE Symp. Series Comput. Intell. (SSCI), 2020, pp. 737–744.

[8] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2017, pp. 23–30.

[9] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization,” in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2018, pp. 3803–3810.

[10] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, “Robot Learning from Randomized Simulations: A Review,” Front. Robot. AI, vol. 9, 2022.

[11] G. Rothermel and M. J. Harrold, “Analyzing Regression Test Selection Techniques,” IEEE Trans. Softw. Eng., vol. 22, no. 8, pp. 529–551, 1996.

[12] S. Yoo and M. Harman, “Regression Testing Minimization, Selection and Prioritization: A Survey,” Softw. Test. Verif. Reliab., vol. 22, no. 2, pp. 67–120, 2012.

[13] J. M. Zhang, M. Harman, L. Ma, and Y. Liu, “Machine Learning Testing: Survey, Landscapes and Horizons,” IEEE Trans. Softw. Eng., vol. 48, no. 2, pp. 1–36, 2022.

[14] V. Riccio, G. Jahangirova, A. Stocco, N. Humbatova, M. Weiss, and P. Tonella, “Testing Machine Learning Based Systems: A Systematic Mapping,” Empir. Softw. Eng., vol. 25, no. 6, pp. 5193–5254, 2020.

[15] J. Zhu, T. Long, and A. Memon, “Automatically Authoring Regression Tests for Machine-Learning-Based Systems,” in Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng.: Softw. Eng. Practice (ICSE-SEIP), 2021, pp. 374–383.

[16] D. Kreuzberger, N. Kühl, and S. Hirschl, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” IEEE Access, vol. 11, pp. 31866–31879, 2023.

[17] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep Reinforcement Learning that Matters,” in Proc. AAAI Conf. Artif. Intell., 2018.

[18] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, “Deep Reinforcement Learning at the Edge of the Statistical Precipice,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2021.

[19] N. A. Lynnerup, L. Nolling, R. Hasle, and J. Hallam, “A Survey on Reproducibility by Evaluating Deep Reinforcement Learning Algorithms on Real-World Robots,” in Proc. Conf. Robot Learning (CoRL), PMLR, vol. 100, 2020.

[20] C. Voloshin, H. M. Le, N. Jiang, and Y. Yue, “Empirical Study of Off-Policy Policy Evaluation for Reinforcement Learning,” in Proc. NeurIPS Datasets and Benchmarks Track, 2021.

[21] Y. Tian, K. Pei, S. Jana, and B. Ray, “DeepTest: Automated Testing of Deep-Neural-Network-Driven Autonomous Cars,” in Proc. 40th Int. Conf. Softw. Eng. (ICSE), 2018, pp. 303–314.

[22] M. Zhang, Y. Zhang, L. Zhang, C. Liu, and S. Khurshid, “DeepRoad: GAN-Based Metamorphic Testing and Input Validation Framework for Autonomous Driving Systems,” in Proc. 33rd IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE), 2018, pp. 132–142.

[23] G. Biggs, “Applying regression testing to software for robot hardware interaction,” in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2010, doi: 10.1109/ROBOT.2010.5509206.

[24] A. Bihlmaier and H. Wörn, “Robot Unit Testing,” in Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR), LNCS vol. 8810, Springer, 2014, pp. 225–236.

[25] F. Lier, S. Schulz, and I. Lütkebohle, “Continuous Integration for Iterative Validation of Simulated Robot Models,” in Simulation, Modeling, and Programming for Autonomous Robots, LNCS vol. 7628, Springer, 2012, pp. 101–112.

[26] M. Mossige, A. Gotlieb, and H. Meling, “Testing robot controllers using constraint programming and continuous integration,” Inf. Softw. Technol., vol. 57, pp. 169–185, 2015.

[27] J. Ernits, E. Halling, G. Kanter, and J. Vain, “Model-based integration testing of ROS packages: A mobile robot case study,” in Proc. Eur. Conf. Mobile Robots (ECMR), 2015.

[28] D. Araiza-Illan, A. Pipe, and K. Eder, “Model-based Test Generation for Robotic Software: Automata versus Belief-Desire-Intention Agents,” in Proc. Int. Symp. Automated Technol. Verif. Anal. (ATVA) Workshops, 2016.

[29] D. Honfi, G. Molnár, Z. Micskei, and I. Majzik, “Model-Based Regression Testing of Autonomous Robots,” in Fundamental Approaches to Software Engineering (FASE), LNCS vol. 10202, Springer, 2017, pp. 119–135.

[30] A. Santos, A. Cunha, and N. Macedo, “Property-based testing for the robot operating system,” in Proc. ACM SIGSOFT Int. Workshop Automating TEST Case Design, Selection, and Evaluation (A-TEST), 2018, pp. 56–62.

[31] M. Abdelgawad, S. McLeod, A. Andrews, and J. Xiao, “Model-based testing of real-time adaptive motion planning (RAMP),” in Proc. IEEE Int. Conf. Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR), 2016.

[32] M. Jiang, K. Miller, D. Sun, Z. Liu, Y. Jia, A. Datta, N. Ozay, and S. Mitra, “Continuous Integration and Testing for Autonomous Racing Software: An Experience Report from GRAIC,” in Proc. IEEE Int. Conf. Robot. Autom. (ICRA) Workshops, 2021.

[33] A. Afzal, D. S. Katz, C. Le Goues, and C. S. Timperley, “Simulation for Robotics Test Automation: Developer Perspectives,” in Proc. IEEE Int. Conf. Softw. Test., Verif. Validation (ICST), 2021, doi: 10.1109/ICST49551.2021.00036.

[34] U. Yayan, “ROSMutation: Mutation Based Automated Testing for ROS Compatible Robotic Software,” Adv. Electr. Comput. Eng., vol. 23, no. 3, 2023.

[35] M. Qu, J. He, Z. Tucakovic, E. Bartocci, D. Ničković, H. Isakovic, and R. Grosu, “DeepRIoT: Continuous Integration and Deployment of Robotic-IoT Applications,” in Proc. ACM/IEEE Int. Conf. Cyber-Phys. Syst. (ICCPS), 2024.

[36] S. O. Sohail, A. Mitrevski, N. Hochgeschwender, and P. G. Plöger, “Property-Based Testing in Simulation for Verifying Robot Action Execution in Tabletop Manipulation,” in Proc. Eur. Conf. Mobile Robots (ECMR), 2021, doi: 10.1109/ECMR50962.2021.9568837.

[37] S. Andersson and G. Carlstedt, “Automated Testing of Robotic Systems in Simulated Environments,” M.S. thesis, 2019.

[38] M. Nguyen, S. Wrede, and N. Hochgeschwender, “Automated Behaviour-Driven Acceptance Testing of Robotic Systems,” in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2025, doi: 10.1109/IROS60139.2025.11246121.

[39] D. Vieira, M. Oliveira, R. Arrais, and P. Melo, “Application of Cloud Simulation Techniques for Robotic Software Validation,” Sensors, vol. 25, no. 6, Art. 1693, 2025.

[40] Y. Jiang, S. Sun, and X. Zheng, “Regression Testing Optimization for ROS-based Autonomous Systems: A Comprehensive Review of Techniques,” arXiv:2506.16101, 2025.

[41] Y. R. Wang, C. Ung, G. Tannert, J. Duan, J. Li, A. Le, R. Oswal, M. Grotz, W. Pumacay, Y. Deng, R. Krishna, D. Fox, and S. Srinivasa, “RoboEval: Where Robotic Manipulation Meets Structured and Scalable Evaluation,” arXiv:2507.00435, 2025.

[42] H. Kress-Gazit, K. Hashimoto, N. Kuppuswamy, P. Shah, P. Horgan, G. Richardson, S. Feng, and B. Burchfiel, “Robot Learning as an Empirical Science: Best Practices for Policy Evaluation,” arXiv:2409.09491, 2024.

[43] J. A. Vincent, H. Nishimura, M. Itkina, P. Shah, M. Schwager, and T. Kollar, “How Generalizable Is My Behavior Cloning Policy? A Statistical Approach to Trustworthy Performance Evaluation,” IEEE Robot. Autom. Lett., vol. 9, no. 10, pp. 8619–8626, 2024.

[44] D. Snyder, A. Badithela, N. Matni, G. J. Pappas, A. Majumdar, M. Itkina, and H. Nishimura, “Beyond Binary Success: Sample-Efficient and Statistically Rigorous Robot Policy Comparison,” in Proc. Robot. Sci. Syst. (RSS), 2026.

[45] A. Badithela, D. Snyder, L. Zha, J. Mikhail, M. O’Kelly, A. Dixit, and A. Majumdar, “Reliable and Scalable Robot Policy Evaluation with Imperfect Simulators,” arXiv:2510.04354, 2025.